Measuring the Cognitive Load of Supervising Parallel AI Coding Agents: A Local, Research-Grounded, Individualized Index

Marinos Prevenios
Independent Research github.com/sagium/ai-code-cognitive-stress

Abstract

Agentic coding tools have placed software developers in a role that was previously confined to specialist operators: one human concurrently supervising several semi-autonomous agents at machine pace, interleaving prompts, judging outputs, and recovering interrupted trains of thought all day. This is a distinct cognitive regime, it accumulates, and existing developer-productivity dashboards do not measure it—they count output, not cognitive cost. We present a method and an open, *private* tool that turns the session logs these tools already generate into a daily, individualized load index. It is meant as a self-assessment aid for an individual engineer—a good-enough proxy for the multitasking overload that precedes burnout—and it deliberately scopes to interaction with coding agents, not the entire workday. The index decomposes into three behaviourally grounded axes—Concurrent Operational Demand Load (CODL), an Interruption Index, and a Closure Deficit—each anchored to an established literature (working-memory capacity, supervisory-control fan-out, the interruption and attention-residue literature, and the Job Demands–Resources model). A central design choice weights concurrency by *engagement*: a session counts at full load only while actively driven and at a discounted weight while "cooking" in the background, a distinction we ground in prospective-memory monitoring costs and the cognition of unfulfilled goals. We deliberately frame the work as a *taskload* measurement (objective demand from events) rather than *workload* (felt experience), state every assumption that is currently unvalidated, and devote a substantial part of the paper to challenging our own construct. We close with falsifiable predictions and a concrete validation roadmap intended to let the community confirm, calibrate, or refute the index.

1 Introduction

Large-language-model coding assistants are routinely run several at a time, or in many concurrent sessions of one. The human operator no longer types most of the code; they *supervise*: dispatching tasks, context-switching between agents, judging partial results, and re-engaging stalled threads. The closest well-studied analogue is supervisory control of multiple semi-autonomous vehicles, where a single operator's performance is known to collapse non-linearly past a personal "fan-out" limit, often *before* the operator consciously registers overload [5, 7, 29].

Three properties make this load hard to see from the inside. First, it is *concurrent*: holding multiple open agent conversations taxes working memory, whose capacity for independent items is famously small (about four) [4]. Second, it is *interruption-dense*: switching between agents and reacting to failures fragments attention and leaves residue that slows the resumed task [15–17, 36]. Third, it is *closure-poor*: agentic work spawns many loops that are parked and resumed rather than finished in one sitting, and each cold resumption reloads decayed context at a cost that grows with the gap [1, 21]. Demands that exceed recoverable resources are the mechanism of burnout in the Job Demands–Resources model [8], and chronic load without recovery—not isolated peaks—is what damages people over time [20, 31].

This is not a speculative concern. For AI-assisted coding specifically, a growing body of peer-reviewed work finds that effort does not disappear under assistance but *shifts* from writing to verification and oversight: programmers spend roughly half their session time in assistant-specific states—verifying suggestions, deferring thought, prompt-crafting, and editing [22]; developers given an AI assistant write less secure code while being *more* confident it is secure [25]; and professional engineers show automation complacency and rising reliance over the course of a single task [27]. A mixed-methods survey of 442 developers links GenAI adoption to burnout through intensified job demands [11]. Crucially, this load is hard to read from the inside: in a randomized trial, experienced developers were slowed by $\approx 19\%$ when allowed AI tooling yet believed it had sped them up [3]—a perception–reality gap echoed in longitudinal telemetry [28] and large-scale developer surveys [33]. A self-perception this miscalibrated is precisely what an honest, behavioural, after-the-fact picture is for.

Our goal is modest and concrete: to give an individual developer an honest, research-grounded, *private* picture of this load over time, scored against *their own* historical baseline rather than a population norm. We do not claim to measure stress, to diagnose burnout, or to replace validated subjective instruments. We claim to make a previously invisible *behavioural* quantity visible, on the user's own machine, with every threshold traceable to the literature.

The intended use is *individual self-assessment*: a working engineer gauging their own multitasking and exhaustion over time, with the index as a good-enough behavioural proxy for the overload patterns that precede burnout—an early warning to act on, not a clinical diagnosis (§6). A scope condition follows from how the signal is built: it sees only the developer's *interaction with coding agents*, not the whole working day. That partial coverage is itself informative. A day whose agent-interaction load is extreme implies one of two things about the rest of that day—either little cognitive capacity remains for other meaningful work, or that work, including the AI-assisted development itself, is degraded by the attention deficit the index is registering. The index is thus better read as *how much of a finite attention budget agent-supervision consumed* than as a tally of activity.

The contributions are:

1. A decomposition of agent-supervision taskload into three axes with explicit, citation-anchored thresholds (§3).

2. An *engagement-weighted* concurrency measure that distinguishes actively-supervised sessions from background "cooking" ones, grounded in prospective-memory and unfulfilled-goal research (§3.3).

3. An open, dependency-free, private reference implementation that serves as a working proof of concept [26] (§4).

4. A deliberately adversarial treatment of our own construct (§6) and a validation roadmap with falsifiable predictions (§7).

2 Background and Related Work

Concurrency and working memory. Cowan's reconsideration places the capacity of focal attention for independent chunks at roughly four [4]; we use this as the reference point past which tracking parallel agents should degrade.

Supervisory control and fan-out. The human-supervisory-control tradition models an operator dividing attention across automated processes [29]. "Fan-out" is the number of robots an operator can effectively manage; it is bounded by *neglect time* (how long a process runs unattended) and *interaction time*, and an agent's *attention demand* is the fraction of operator time it consumes [5, 7]. This fraction framing directly motivates our engagement weighting (§3.3).

Interruption and attention residue. Interrupted work is completed faster but at the cost of higher stress and effort [17]; fragmented work and frequent external switches are costly [16], with residue from an unfinished task impairing the next [15], and cross-modality or cross-tool switches costlier than within-tool ones [36].

Demands, recovery, and the optimum. The Job Demands–Resources model frames burnout as demands outstripping recoverable resources [8]; allostatic-load theory stresses that *sustained* activation, not peaks, causes harm [20]; and psychological detachment in off-hours predicts next-day well-being [31, 32]. Performance follows an inverted-U in arousal/load [37], the "flow channel" between boredom and anxiety [6], motivating a *personal optimum* rather than a fixed ceiling.

Open loops and monitoring costs. Two literatures ground the claim that a backgrounded task is neither free nor full cost. Unfulfilled goals remain active in memory and intrude on unrelated tasks until the goal is closed or a concrete plan is made [18]; and maintaining a pending intention while doing other work measurably slows that ongoing task—the prospective-memory "monitoring cost," on the order of 15–20% of baseline [30].

AI-assisted coding and developer cognition. A young but fast-growing empirical literature studies these tools directly, and it is what makes the present concern concrete rather than analogical. AI assistance shifts effort toward verification: programmers spend ≈51.5% of session time in suggestion-specific states [22], and merely *informing* developers that code is LLM-generated raises their measured NASA-TLX workload [34]. Over-reliance is demonstrated rather than hypothesised—a "false sense of security" among assistant users who write less secure but more confidently-trusted code [25], and automation complacency among professional engineers [27]—and the interaction is bimodal, alternating fast *acceleration* with slow, validation-heavy *exploration* [2]. Burnout now has its first peer-reviewed anchor in a Job Demands–Resources analysis of GenAI adoption [11]. What this literature still largely lacks is direct, longitudinal, AI-vs-no-AI measurement of cognitive load in everyday professional work [28]; our index approaches that gap from the opposite direction—an objective behavioural signal computed continuously from real sessions on the developer's own machine, rather than a one-off laboratory measurement.

What we are *not*. The validated instrument for subjective workload is NASA-TLX [14]; for diagnosed

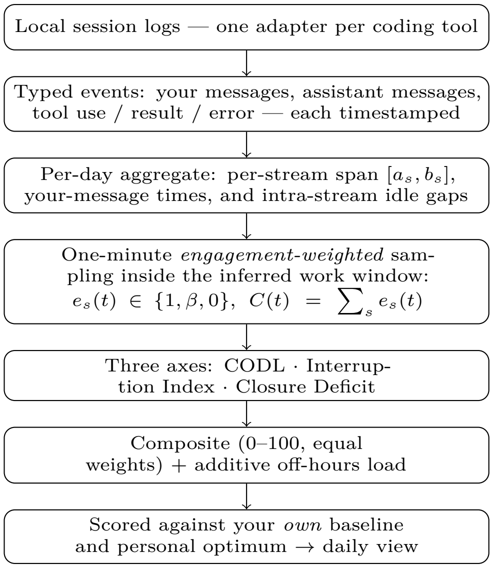

Figure 1: From logs to a daily score, entirely on the user's machine. Overlapping sessions are swept at one-minute resolution and weighted by engagement (foreground 1, background β), so raw concurrency becomes the engagement-weighted load $C(t)$ (§3.3); the three axes and composite are evaluated per day and read only against the individual's own history.

burnout it is the Maslach Burnout Inventory [19]. Our index is neither; it is an objective behavioural proxy, and §6 treats the gap as the central threat.

3 Method

The pipeline from raw logs to a daily score is summarised in Figure 1; the remainder of this section defines each stage.

3.1 Data model

Input is the set of local session transcripts a coding tool already writes. A pluggable adapter maps each tool's log into a common vocabulary of typed events (user message, assistant message, tool use, tool result/error) with UTC timestamps. Events are reduced to per-day *stream* records: one stream is one session on one day, summarised by its first/last event timestamps $[a_s, b_s]$, per-type counts, and the multiset of the user's own message timestamps U_s. All processing is local; nothing is transmitted (§4).

3.2 Work window

Axes are computed inside a daily work window $W = [w_0, w_1]$ *inferred per operator*: $w_0 = \lfloor p_{10} \rfloor$ and $w_1 = \lceil p_{90} \rceil$ are the p_{10} and p_{90} of the local-time hours at which the operator sends messages, floored and ceiled *outward* to whole hours so the band is never narrower than the percentile span, pooled over the baseline window and applied as one stable band to *every* calendar day (a degenerate or empty band falls back to the cold-start default below). The window is therefore individual to the operator rather than a property of the experiment. We make no assumption about which days are working days—there is no privileged weekend; a person can work any day. "Work" is whatever falls inside the operator's own inferred hours, on any date; engaged activity past the end of those hours—or implausibly far before their start—is the off-hours *recovery* signal, again on any date (the boundary's asymmetry is defined with the composite below). A manual band may be pinned in configuration to override inference; before enough data accrues (< 5 distinct dates of activity) a conventional 09:00–19:00 local band serves as a cold-start default. We sample the window at one-minute resolution; let T be the set of sample instants.

Off-hours is thus defined relative to the operator's own agent-interaction hours, not the calendar: the index measures the *additional* strain of agentic coding, not total workload or work–life balance. It follows that the index has no notion of a *rest day*—a seven-day-a-week operator with consistent hours registers no off-hours signal—which is a scope boundary rather than a defect.

3.3 Engagement-weighted CODL

The naive concurrency count treats a stream as one continuously-active locus across its whole $[a_s, b_s]$ span. This over-counts: a session left running while the operator is away reads identically to one being actively juggled. We instead weight each stream by *engagement* at each instant. Let g be a foreground grace window (default 5 minutes) and $\beta \in [0, 1]$ a background weight. The instantaneous weight of stream s at time t is

$$e_s(t) = \begin{cases} 1 & \text{if } \exists\, u \in U_s : \ 0 \le t - u \le g \\ \beta & \text{else if } a_s \le t \le b_s \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

That is, a session counts fully only within g of one of the operator's own messages (*foreground*, actively driven); otherwise, while alive, it is "cooking" in the background at weight β. Weighted concurrency and the two CODL summaries are

$$C(t) = \sum_s e_s(t), \quad \overline{\mathrm{CODL}} = \frac{1}{|T|} \sum_{t \in T} C(t),$$
$$\mathrm{CODL}^{\star} = \max_t C(t). \tag{2}$$

We retain a raw headcount $H(t) = \sum_s \mathbf{1}[a_s \le t \le b_s]$ and its peak $\max_t H(t)$ as a descriptive "sessions open at once," but the *scored* quantities use the weighted form.

Choosing β. A background locus is neither free nor full cost. A session left "cooking" while the operator attends elsewhere is precisely the *out-of-the-loop* regime of human supervisory control: the operator periodically lets a semi-autonomous process run unattended and must reengage to judge or correct it. That literature is consistent on three points that bear directly on β. Monitoring a process you have let run is not idle time but *effortful*, and sustained monitoring is itself a source of stress [35]; remaining out of the loop degrades situation awareness and imposes a cost when control is resumed [9,10]; and under concurrent load operators systematically under-monitor automation (complacency), so the divided-attention cost is real rather than hypothetical [23]. Together these ground a positive residual cost—$\beta > 0$—in a regime far closer to ours than the laboratory. For the *magnitude* we still borrow a number: the prospective-memory cost of holding a pending intention, ≈15–20% of ongoing-task capacity [30], with unfulfilled goals consuming working memory until closed [18], and the supervisory-control "attention demand" fraction on the operator side [5]. We set $\beta = 0.25$, the conservative top of that band, erring toward a stronger anti-fire-and-forget signal. β and g are configuration parameters, not hard-coded constants, so the assumption is auditable and tunable (§7).

3.4 Interruption Index

Tool calls *within* an active session are not counted: while an agent works, the operator is in a "waiting" state in which attention can legitimately detach, so counting each call would inflate the rate by orders of magnitude and violate the standard definition of an interruption as an unscheduled event demanding immediate attention. Two events do pull attention: a tool *error* (the operator may need to intervene) and a *cross-stream start* (a new session lit up while another was active, forcing a switch, with attention residue [15]). With E^{win} the tool-error count apportioned to the work window, X the in-window cross-stream starts, and h the window length in hours,

$$I = \frac{1.5\,E^{\text{win}} + 3.0\,X}{h}. \qquad (3)$$

The weights follow the relative cost of external versus self-interruption in the interruption literature [16, 17, 36]; the normalization ceiling is 10/hour, well above the field baseline of a working-sphere switch roughly every ≈11 minutes (≈5/hour) observed in knowledge work [13].

3.5 Closure Deficit

A loop you cannot finish in one sitting and have to pick back up later is a loop left open, and reopening it is not free: a suspended goal's activation decays over the gap and must be reconstructed on return [1], and the cost of that reconstruction rises with how long the task was suspended [21]. In the programming domain specifically, resuming an interrupted task reliably incurs a context-reconstruction tax—only a small fraction of interrupted sessions resume coding within a minute, and fewer still resume without first navigating to rebuild context [24]. Closure, conversely, is a recovery resource [32], and the tendency to return to interrupted tasks is a robust behavioural regularity, even as the older claim that open tasks are better *remembered* fails to replicate [12]. We therefore measure the day's *resumption load*: how much the operator re-entered parked sessions, weighted by how cold each had gone.

A *resume* is a true-idle gap in a single session's event timeline—a span during which no event of any kind (user message, model message, or tool call) was recorded—of at least a threshold τ ($\tau = 30\,\text{min}$ by default; below it the loop was paused, not suspended). A session continued on a later calendar day contributes a cross-day resume on the day work resumes. The cost of one resume saturates with the gap length g,

$$\sigma(g) = \min\Big(1,\ \frac{g}{\Delta}\Big), \qquad (4)$$

where Δ (= 120 min) is the horizon past which the context is treated as fully cold, so a six-hour gap is no costlier than a two-hour one once activation has fully decayed—the linear-then-flat shape of the duration–cost relationship [1, 21]. With $\mathcal{R}$ the set of resumes whose pickup instant falls inside the work window W, and a daily ceiling K (= 4 fully-cold reloads, a loose working-memory anchor [4]: cold- reloading more than about four distinct loops in a day means thrashing across more open contexts than working memory comfortably holds),

$$\text{ClosureDeficit} = \text{clip}\Big(\frac{1}{K}\sum_{g\in\mathcal{R}} \sigma(g),\ 0,\ 1\Big). \qquad (5)$$

A value of 0 is a real, good score—every loop was closed in one sitting—and the axis is defined on every active day, requiring no configuration. It is *undefined* (omitted as data, not scored) only on a day with no activity at all. The thresholds τ, Δ, and K are citation-anchored modeling priors, not values fitted to felt load; all three are configurable.

This axis is built from per-session resume *events*, not from the concurrency time-series $C(t)$, so it is independent of the CODL shape by construction: two days with identical $C(t)$ (hence identical $\overline{\text{CODL}}$) receive different deficits if one parked-and-reloaded its loops while the other ran them to completion in a sitting—information $C(t)$ cannot carry. The gap is, however, a *proxy* for a parked loop and not a certainty (§6): a long *autonomous* agent turn is correctly excluded because it is not idle, but stepping away with a session still mid-turn is not distinguished from a true suspension. And the supporting laboratory evidence [1, 21] measured *short* interruptions, on the order of seconds to a minute; the multi-hour and cross-day gaps this axis scores extrapolate well beyond that regime, with the field study of interrupted programming [24] as the closest in-domain bridge.

3.6 Composite

Each available axis is normalized to $[0, 1]$ and blended with equal weights. When the Closure Deficit is defined for the day, all three contribute:

$$\text{Composite} = \frac{100}{3}\Bigg(\min\Big(1, \frac{\overline{\text{CODL}}}{5}\Big) + \min\Big(1, \frac{I}{10}\Big) + \text{clip}(\text{ClosureDeficit}, 0, 1)\Bigg). \quad (6)$$

The Closure Deficit is defined on every active day (§3.5) and is undefined only on a day with no activity at all; on such a day the blend *drops* that axis and renormalises over the remaining two rather than imputing a 0. In general the composite is the weight-normalised mean over whichever axes are defined for the day. The CODL ceiling is set to 5—one above Cowan's ≈ 4 capacity limit—so a day operating *at* the working-memory boundary scores 0.8 (the warning band) rather than saturating the axis at 1.0. Off-hours work then adds an *additive* load to this base. Let o be the day's off-hours *engaged* minutes—one-minute instants during which the operator was actively driving a session (within the foreground grace g of one of their own messages, §3.3) that fall *after* the end of W, or more than an early-start grace e (default 3 hours) *before* its start. The reported composite is

$$\text{Composite}^{*} = \min\big(100,\ \text{Composite} + 30 \cdot \min(1, o/90)\big). \quad (7)$$

Three design choices matter here. First, the toll is *additive*, not multiplicative: off-hours work always counts, so a day worked *entirely* outside the window—zero in-window base—still scores a positive composite, which a multiplicative form would wrongly leave at zero. Second, o is anchored to *interaction*, not stream liveness: a background session left running while the operator is away contributes nothing—only the operator actually being present and driving the agent outside their own hours counts (e.g. waking in the night to check on a run). Third, the boundary is *asymmetric*: every engaged minute past the window end counts, while minutes before its start count only beyond the early-start grace. The recovery literature locates the cost in failure to *disengage* at the day's end [32]; beginning somewhat earlier than usual is ordinary schedule shift, not a recovery deficit, and penalising it would punish the healthy direction of flexibility. The grace is bounded so that genuinely nocturnal engagement stays visible: small-hours work (e.g. 01:30 against a noon start) lies far beyond any plausible early start and counts in full. The inferred band W is the operator's norm; engaged work past it—or outlier-early before it—is treated as a deviation from that norm. Like the background weight β, the magnitudes (the +30-point toll saturating at 90 engaged minutes, and the 3-hour grace) are explicit priors, not fitted values (§6), grounded in disengagement as a recovery resource [32]. Equal weights are the explicit *null hypothesis*: we have no evidence that one axis dominates felt load, and we prefer to state that than to fit weights to a single user's data. Replacing equal weights with empirically calibrated ones is a primary validation target (§7). The reference implementation colors and ranks each day against the operator's *own* percentile distribution (p50/p75/p90) computed over active days in the baseline window—never a population norm—reinforcing that only relative movement within one person's history is interpreted (see the corresponding caveat in §6).

3.7 Personal optimum and recovery

Performance follows an inverted-U with load [6, 37]. We bucket historical workdays by $\overline{\text{CODL}}$ (width 0.5) and score each bucket by closure achieved against off-hours follow-up; the midpoint of the best bucket is reported as the operator's individual "flow channel" target, a line on the chart rather than a ceiling. Because the Closure Deficit is independent of concurrency (§3.5), this scoring term is not a function of the very $\overline{\text{CODL}}$ used to form the buckets, so the optimum is not circular: it asks which load band *actually* closed the most loops, rather than rediscovering the load band with the least concurrency. It requires at least 14 active days to stabilise, below which the report shows "calibrating." Off-hours activity (on any date) is flagged because sustained load without detachment, not peaks, predicts harm [20, 31, 32].

3.8 Method revision: the paper as adversarial reviewer

The method above is not static; it is the current state of a deliberate revision loop (Figure 2) in which the paper and the implementation co-evolve under human direction. The tool was built first, as the concrete artifact this paper documents (§4). Writing the paper then forced each assumption to be stated precisely enough to attack, and the resulting self-critique (§6) became the *driver* of subsequent change rather than a closing disclaimer: each threat is read as an adversarial review of the code and of the premise behind an axis.

A single pass takes one self-critique and first *verifies it against the running code*—does the weakness actually exist, and is it described accurately?—then either corrects the paper where it mis-states its own artifact, or changes the method and code to remove the weakness and rewrites the affected §3 and §6 text to match, or both. The work is carried out with agentic coding assistants—the same class of tool this paper measures—under a human-in-the-loop gate: an agent investigates and proposes a plan; the human approves it before any code or prose is touched; an agent implements; the test suite must stay green; the paper is rebuilt; and the human reviews the

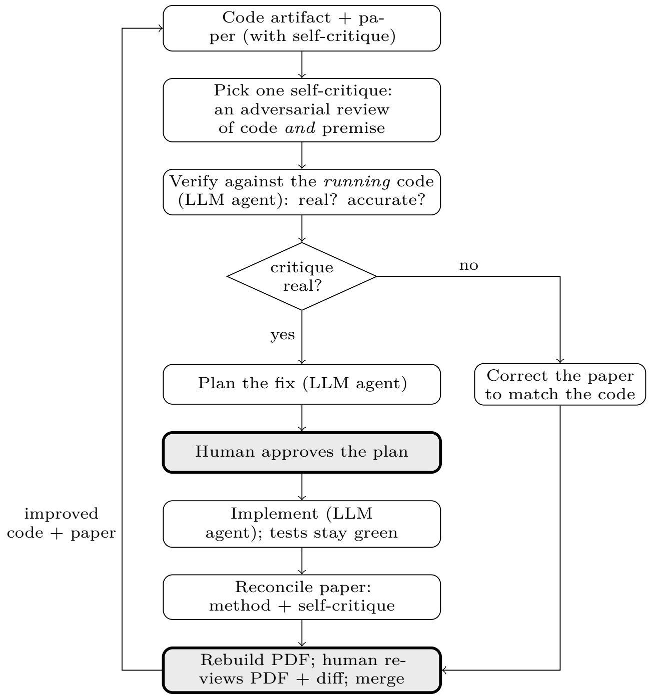

Figure 2: The `reconcile-critique` loop (§3.8): the paper's self-critique drives revision of an *executable* artifact. Two human gates (shaded) bracket every change—plan approval and merge review. Verification and planning use one LLM agent; implementation a separate one. A pass that finds the critique unfounded corrects the paper instead of the code.

rendered result and the diff before the change is accepted. Several axes reached their current form this way—the per-operator inferred work window, the interaction-anchored off-hours load, the removal of the hard-coded weekend, and the in-domain grounding of the background weight β each began as a §6 critique.

Two properties make this more than ordinary editing. First, the artifact is *executable*, so a critique of the premise is checkable: if the paper claims the code does X, that is either true of the running code or it is a defect in one of them, and the loop must resolve the disagreement rather than soften the wording. Second, the process is *reflexive*—a paper about the cost of supervising agentic coding is itself produced by supervising agentic coding—so the authors are incidentally n=1 subjects of the regime the index scores (§6). We make the loop explicit because it, more than any single threshold, is the method's route to eventually being right: the index is built to be falsified and revised, and §7 states the external evidence meant to drive later passes.

4 Implementation

The method is realised in an open-source reference implementation [26] that doubles as the proof of concept for this paper: every axis, threshold, and the engagement weighting described here is exactly what the released tool computes.[1] It is pure Python standard library (no third-party runtime dependencies), which keeps the trust surface small and the install trivial. It reads only local files, performs no network I/O at any stage, and writes a self-contained HTML report plus a machine-readable JSON sibling. Per-day aggregates are cached keyed by source-file modification time, so metric or weighting changes recompute from cache without re-parsing logs. Adding a new coding tool is a single adapter implementing the event protocol; the aggregate, metric, optimum, and rendering stages are tool-agnostic. The tool is private by design: session transcripts frequently contain proprietary code and secrets, so the generated artefacts are treated as similarly sensitive and never leave the machine.

The implementation also provides two optional, private pathways toward the multi-developer calibration of §7. The first is an anonymized, consent-gated *research export*: a per-user file of derived daily metrics (and their components) with the timezone dropped and calendar dates randomly shifted, which the operator may choose to upload manually—the tool itself transmits nothing, preserving the privacy invariant. The second is a population *calibration* step that pools such exports and proposes normalization ceilings and composite weights that cover the observed range of work patterns. Both read and write only local files. The calibration is *unsupervised*: it fits the axis *scales* to the population distribution and suggests weights from the redundancy among axes, but it does not fit weights to felt load—that still requires a subjective criterion (§7).

5 Illustrative Example

To make the axes concrete we report one author's own data for a single month (19 active workdays). We stress that this is an *illustration of the instrument*, not evidence of validity; n=1, the subject is an author, and there is no ground-truth load measure here. Qualitatively, the engagement-weighted CODL pulled composite scores down on days dominated by background "cooking" sessions while leaving interruption-dominated days largely unchanged, and the weighted peak CODL* separated a day of genuine four-way *active* supervision from a day with four sessions merely open. This is the behaviour the design intends; whether it tracks *felt* load is exactly the open question of §7.

[1] Source and documentation: `https://github.com/sagium/ai-code-cognitive-stress`. The repository is being prepared for public release; the qualitative behaviour described in this section was produced by the released code and is reproducible from a user's own logs.

6 Threats to Validity and Self-Critique

We consider the following the most serious challenges to the method, roughly in order of severity. We see no benefit in understating them.

Taskload is not workload. Every axis is computed from objective event streams; none measures felt experience. Correlation with felt overload is only moderate, and objective and subjective workload are known to dissociate. A calm composite does not prove the operator felt calm, and vice versa. The validated subjective instrument is NASA-TLX [14]; until we correlate against it (or experience sampling), the index is an unvalidated proxy for the construct it names.

The supervisory-control analogy is borrowed, not established. Fan-out, neglect time, and attention demand were developed for UAV/robot operators [5, 7]. Their transfer to LLM oversight is plausible—both involve one human time-sharing across semi-autonomous processes—but unproven. LLM agents differ in failure modes, feedback latency, and the verbal/code modality of interaction. Recent AI-coding studies give the transfer partial, in-domain support: automation complacency and rising reliance have now been observed directly in professional developers using LLM assistants [27], and informing developers that code is LLM-generated measurably raises their NASA-TLX workload [34]. These vary trust and provenance rather than the supervisory fan-out our axes model, so they corroborate the cognitive cost without yet validating the specific fan-out/neglect-time constructs we borrow.

The background weight β is a prior, calibrated only in magnitude. That a background locus carries a positive, effortful, and stress-inducing monitoring cost is supported *in domain* by the supervisory-control and automation literature on out-of-the-loop operation, vigilance, and complacency (§3.3) [9,10,23,35]—a regime far closer to agent supervision than the laboratory. What remains imported is the *magnitude*: the specific 15–20% figure behind $\beta = 0.25$ comes from prospective-memory and goal-priming paradigms [18,30], not from software supervision, so the value is a defensible prior rather than a measurement. The true β may differ by person, task, and tool, and may not be constant within a day; recovering it from data is a validation target (§7).

Equal composite weights are a placeholder. We assert no axis dominates; this is almost certainly false in detail. Without calibration against a criterion, the composite's absolute scale is arbitrary and only its *relative* movement within one person's history is meaningful. The weights are a configuration parameter rather than a hard-coded constant, and the implementation includes a population step that suggests data-fitted weights from the redundancy among axes across pooled exports (§4). That step is *unsupervised*, however: absent a felt-load criterion it cannot show the fitted weights track subjective load, so equal weights remain the default and the placeholder stands until the supervised calibration of §7.

The Closure Deficit's idle gap is a proxy for a parked loop. The axis scores *resumption load*: idle gaps in a session's timeline where a parked loop was picked back up, weighted by gap length (§3.5). This makes it independent of concurrency—two days with identical $C(t)$ score differently if one reloaded its loops cold and the other did not. The axis has its own limitations, which we state plainly. First, the gap is a *proxy*: it is genuine wall-clock dormancy (no event of any kind), so a long *autonomous* agent turn is correctly *not* counted as a resume, but the converse failure remains—stepping away while a session is mid-turn, or leaving a session idle in the foreground, is indistinguishable from a deliberate suspension, and a single prompt that triggers a very long tool run slightly over-attributes the agent's working time to the parked span. Second, and most importantly, the duration→cost relationship this axis leans on was measured in the laboratory over *short* interruptions—seconds to about a minute [1, 21]; the multi-hour and cross-day gaps that dominate real agentic work extrapolate well beyond that regime. The field study of interrupted programming [24] is the closest in-domain anchor that resumption is costly at the minutes-and-up scale, but the specific severity curve $\sigma(g) = \min(1, g/\Delta)$ and the saturation horizon Δ are a modeling choice, not a fitted function. Third, the three constants—the resume threshold τ, the horizon Δ, and the daily ceiling K—are citation-anchored priors, not values calibrated against felt load; the ceiling in particular borrows Cowan's working-memory bound by analogy rather than measurement. A retrieval-cue effect we do *not* model cuts the other way and is worth noting: an agent transcript is itself a strong cue for reconstructing a suspended goal [1], so a resumed *agent* session may be cheaper to reload than the cue-free laboratory task the curve is drawn from—meaning the axis, if anything, may over-state the cost of a resume. The axis is defined on every active day and returns no value only when there is no activity at all (§3.6).

Engagement is interaction-anchored by design. Foreground is proxied by proximity to the operator's own messages within a grace window g. Silent engagement (reading, thinking, reviewing diffs without typing) leaves no event and is therefore not counted—but this is a scope boundary by construction, not a measurement gap (§3): the index measures the load *driven by agent interaction*, not total cognitive work. The consequence is conservative in the direction that matters. Anchoring to logged interaction can only *understate* the operator's true cognitive load, so the measured agent-supervision load is a lower bound on total load: a high score is a sufficient indicator of overload regardless of whatever silent work goes unrecorded, because if agent supervision alone saturates capacity there is no headroom left for the rest. The one bias in the other direction is that any operator message—including a perfunctory acknowledgement—grants full foreground weight for the whole window, so a trivial message can be over-weighted by up to g. The 5-minute grace is a default, not fitted; like β it is a configurable, auditable parameter rather than a hidden constant, and both the value of engagement weighting and the sensitivity to g are validation targets (§7).

Cross-tool comparability is unestablished. Different tools log at different granularities (e.g. per-keystroke vs. per-turn), so event counts and stream boundaries are not commensurable across adapters. Aggregating or comparing across tools without normalization risks systematic bias.

Single subject and borrowed thresholds. The capacity threshold (≈ 4 [4]) and interruption ceiling (10/hour) are literature values, not fitted to the population of agent-coding developers, who may differ. All reported behaviour is n=1 and from an author. The reference implementation provides the machinery to move beyond this—an anonymized per-user export and a population-calibration step that refits the ceilings to the pooled distribution (§4)—so what remains is collecting the multi-developer sample, each participant running the private tool on their own machine and only derived metrics pooled. Until that sample exists the thresholds stay borrowed and the data stays n=1 (§7).

Modelling and boundary choices. One-minute sampling, the over-approximation of a stream as alive across $[a_s, b_s]$, and timezone handling are all defaults that could materially move scores for shift workers or bursty users. Timezone is always the operator's own local zone and the tool is single-subject by design, so cross-timezone teams are out of scope rather than a threat to this index. The work window is inferred per operator from the p_{10}–p_{90} of message hours rather than fixed to a universal band, which avoids assuming one schedule fits every operator but introduces its own choices—the percentile edges, the hour rounding, and the five-date minimum before inference engages—and assumes that message timing tracks actual working hours; an operator who works silently or messages at odd hours will get a skewed band, and inference falls back to the fixed default during cold start. The additive off-hours load that lets work past the band raise the composite is likewise built from explicit priors (+30 points, saturating at 90 engaged minutes; a 3-hour early-start grace on the band's lower edge), not measured penalties; the magnitudes could over- or understate the true cost of off-hours work—and the grace could misclassify a habitual pre-dawn shift as outlier work or vice versa—so both are targets for calibration. That load is anchored to *interaction*—engaged minutes within the grace of the operator's own messages—so it ignores background sessions running while the operator is away and counts only active supervision of the agent. Silent reading or review without typing, and the lack of any "rest day" notion, are therefore out of scope by construction rather than measurement gaps: the index targets the additional strain of *agentic coding* surfaced through interaction, not total work or work–life balance (§3). Additionally, tool errors are apportioned to the work window by assuming they are uniformly distributed across each stream's $[a_s, b_s]$ lifetime—a defensible default since the per-day aggregate carries no per-error timestamps, but one that could misattribute errors from bursty or session-boundary activity.

Construct and causal claims. "Stress" in the project name is aspirational; the composite is a load index. No causal link to burnout outcomes (e.g. MBI [19]) is claimed or shown; the index is offered as a self-assessment proxy for the overload patterns *associated* with burnout risk—an early-warning aid—not a validated burnout measure. Reactivity is also possible: surfacing the score may itself change behaviour. And because the index covers only coding-agent interaction (§3), it is silent on total workload: a low score does not certify a sustainable day, only a light *agent-supervision* load.

7 Falsifiable Predictions and Validation Roadmap

The above critiques are not fatal; they are a research agenda. We state predictions that would, if tested, confirm or refute the design.

1. **Concurrent validity.** Across a sample of agent-coding developers, daily composite should correlate positively with same-day NASA-TLX [14] or ecological momentary assessments of effort. A null or negative correlation falsifies the composite as a load proxy.

2. **Engagement-weighting ablation.** Engagement-weighted CODL should correlate with felt load *better* than the flat headcount. If weighting does not improve the correlation, β and g add complexity without value.

3. **Weight calibration.** Regressing subjective load on the three normalized axes should yield weights that beat the equal-weight null out-of-sample; the fitted weights become the next version's defaults. The implementation already derives *unsupervised*,

redundancy-based weight suggestions from pooled exports (§4); this prediction concerns the *supervised* step the tool cannot perform—fitting and validating weights against a felt-load criterion.

4. **Predictive validity for recovery.** Days high in off-hours activity and composite should predict lower next-day vigor/detachment scores [31]; absence of this relationship weakens the recovery framing.

5. β **recovery from data.** With per-event timestamps and a secondary signal (e.g. self-reported "which sessions were you actually watching"), β can be estimated rather than assumed; the estimate's distance from 0.25 quantifies our prior's error.

6. **Sensitivity analysis.** Composite rankings of days should be stable under reasonable perturbations of g, the sampling interval, and the thresholds; instability would indicate over-tuned, non-robust scoring.

7. **Inferred window vs. fixed band.** The per-operator inferred window should predict felt load (or in-flow time) better than the fixed 09:00–19:00 band out-of-sample. If inference does not beat the fixed band, the percentile machinery adds complexity without value and the fixed default should return.

8. **Off-hours load validity.** Days carrying a high off-hours engaged-minutes load should predict lower next-day vigor/detachment [31]; if the load does not track reduced recovery, its magnitude (+30 points, 90-minute saturation) is mis-specified and should be refit or dropped.

9. **Population threshold check.** With multi-developer data, the ≈ 4 capacity ceiling [4] and 10/hour interruption ceiling can be checked and, if warranted, refit to the agent-coding population; large divergence would mean the borrowed values mis-set the axis scales. The calibration step computes exactly this from pooled exports (§4); what remains is collecting the population.

A modest multi-developer study with experience sampling—each participant running the private tool on their own machine, with only the derived per-day metrics and subjective ratings pooled for analysis, so no off-machine data path is added to the tool—is supported end-to-end by the released tooling (the anonymized export and the pooled-calibration step, §4) and is the primary outstanding work; running it would address the majority of the threats in §6, in particular the single-subject and borrowed-threshold limitations, converting this from a principled instrument into a validated one. On the closure side, the outstanding work is calibrating the resume threshold, decay horizon, and daily ceiling against felt load rather than borrowing them (§3.5), and better separating a deliberately parked loop from an idle foreground session.

8 Ethics and Privacy

The tool reads proprietary source and potentially secrets; we therefore treat private processing as a hard invariant, not a feature, and any future data-sharing path would require separate, explicit, opt-in consent. The intended use is individual self-reflection. We explicitly warn against managerial or comparative use across people: the index is calibrated to an individual's own history, the construct is unvalidated, and using an unvalidated load proxy for evaluation would be both statistically and ethically unsound.

9 Conclusion

Supervising parallel AI coding agents is a new, accumulating, and largely unmeasured cognitive regime. We have presented a local, privacy-preserving, literature-anchored method to make its *behavioural* footprint visible to the individual who generates it, including an engagement-weighted concurrency measure that distinguishes active supervision from background cooking. We have been deliberate about what the method does not yet establish, and have laid out falsifiable predictions and a validation path. We hope the explicit assumptions and the open implementation make the construct easy to challenge—which is the fastest way to make it, and any tool built on it, more trustworthy.

References

[1] Erik M. Altmann and J. Gregory Trafton. Memory for goals: an activation-based model. *Cognitive Science*, 26(1):39–83, 2002.

[2] Shraddha Barke, Michael B. James, and Nadia Polikarpova. Grounded copilot: How programmers interact with code-generating models. *Proc. ACM on Programming Languages (OOPSLA)*, 7(OOPSLA1), 2023.

[3] Joel Becker, Nate Rush, Beth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. METR, 2025. arXiv:2507.09089; preprint, not peer-reviewed.

[4] Nelson Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.

[5] J. W. Crandall and M. L. Cummings. Identifying predictive metrics for supervisory control of multiple robots. *IEEE Transactions on Robotics*, 23(5):942–951, 2007.

[6] Mihaly Csikszentmihalyi. *Flow: The Psychology of Optimal Experience*. Harper & Row, 1990.

[7] M. L. Cummings and P. J. Mitchell. Predicting controller capacity in supervisory control of multiple UAVs. *IEEE Transactions on Systems, Man, and Cybernetics — Part A*, 38(2):451–460, 2008.

[8] Evangelia Demerouti, Arnold B. Bakker, Friedhelm Nachreiner, and Wilmar B. Schaufeli. The job demands–resources model of burnout. *Journal of Applied Psychology*, 86(3):499–512, 2001.

[9] Mica R. Endsley and Esin O. Kiris. The out-of-the-loop performance problem and level of control in automation. *Human Factors*, 37(2):381–394, 1995.

[10] Alexander Eriksson and Neville A. Stanton. Takeover time in highly automated vehicles: Noncritical transitions to and from manual control. *Human Factors*, 59(4):689–705, 2017.

[11] Zixuan Feng, Sadia Afroz, and Anita Sarma. From gains to strains: Modeling developer burnout with GenAI adoption. In *Proc. 48th Int. Conf. on Software Engineering, Software Engineering in Society Track (ICSE-SEIS '26)*, 2026. arXiv:2510.07435.

[12] Romain Ghibellini and Beat Meier. Interruption, recall and resumption: a meta-analysis of the zeigarnik and ovsiankina effects. *Humanities and Social Sciences Communications*, 12(1):1–13, 2025.

[13] Victor M. González and Gloria Mark. "constant, constant, multi-tasking craziness": Managing multiple working spheres. In *Proc. SIGCHI Conf. on Human Factors in Computing Systems (CHI '04)*, pages 113–120, 2004.

[14] Sandra G. Hart and Lowell E. Staveland. Development of NASA-TLX (task load index): Results of empirical and theoretical research. In *Human Mental Workload*, volume 52 of *Advances in Psychology*, pages 139–183. North-Holland, 1988.

[15] Sophie Leroy. Why is it so hard to do my work? the challenge of attention residue when switching between work tasks. *Organizational Behavior and Human Decision Processes*, 109(2):168–181, 2009.

[16] Gloria Mark, Victor M. Gonzalez, and Justin Harris. No task left behind? examining the nature of fragmented work. In *Proc. SIGCHI Conf. on Human Factors in Computing Systems (CHI '05)*, pages 321–330, 2005.

[17] Gloria Mark, Daniela Gudith, and Ulrich Klocke. The cost of interrupted work: more speed and stress. In *Proc. SIGCHI Conf. on Human Factors in Computing Systems (CHI '08)*, pages 107–110, 2008.

[18] E. J. Masicampo and Roy F. Baumeister. Consider it done! plan making can eliminate the cognitive effects of unfulfilled goals. *Journal of Personality and Social Psychology*, 101(4):667–683, 2011.

[19] Christina Maslach and Susan E. Jackson. The measurement of experienced burnout. *Journal of Occupational Behaviour*, 2(2):99–113, 1981.

[20] Bruce S. McEwen. Protective and damaging effects of stress mediators. *New England Journal of Medicine*, 338(3):171–179, 1998.

[21] Christopher A. Monk, J. Gregory Trafton, and Deborah A. Boehm-Davis. The effect of interruption duration and demand on resuming suspended goals. *Journal of Experimental Psychology: Applied*, 14(4):299–313, 2008.

[22] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. In *Proc. CHI Conf. on Human Factors in Computing Systems (CHI '24)*, 2024.

[23] Raja Parasuraman and Dietrich H. Manzey. Complacency and bias in human use of automation: An attentional integration. *Human Factors*, 52(3):381–410, 2010.

[24] Chris Parnin and Spencer Rugaber. Resumption strategies for interrupted programming tasks. *Software Quality Journal*, 19(1):5–34, 2011.

[25] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do users write more insecure code with AI assistants? In *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS '23)*, pages 2785–2799, 2023.

[26] Marinos Prevenios. ai-code-cognitive-stress: a local, research-grounded cognitive-load profiler for agent-coding tools. Open-source software, 2026. Reference implementation / proof of concept. `https://github.com/sagium/ai-code-cognitive-stress`.

[27] Crystal Qian and James Wexler. Take it, leave it, or fix it: Measuring productivity and trust in human–AI collaboration. In *Proc. 29th Int. Conf. on Intelligent User Interfaces (IUI '24)*, 2024.

[28] Agnia Sergeyuk, Eric Huang, Dariia Karaeva, Anastasiia Serova, Yaroslav Golubev, and Iftekhar Ahmed. Evolving with AI: A longitudinal analysis of developer logs. In *Proc. 48th Int. Conf. on Software Engineering (ICSE '26)*, 2026.

[29] Thomas B. Sheridan. *Telerobotics, Automation, and Human Supervisory Control*. MIT Press, 1992.

[30] Rebekah E. Smith. The cost of remembering to remember in event-based prospective memory: Investigating the capacity demands of delayed intention performance. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 29(3):347–361, 2003.

[31] Sabine Sonnentag, Carmen Binnewies, and Eva J. Mojza. Staying well and engaged when demands are high: The role of psychological detachment. *Journal of Applied Psychology*, 95(5):965–976, 2010.

[32] Sabine Sonnentag and Charlotte Fritz. The recovery experience questionnaire: Development and validation of a measure for assessing recuperation and unwinding from work. *Journal of Occupational Health Psychology*, 12(3):204–221, 2007.

[33] Stack Overflow. 2025 developer survey. `https://survey.stackoverflow.co/2025/`, 2025. Industry survey, 49,000+ respondents; not peer-reviewed.

[34] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. Developer behaviors in validating and repairing LLM-generated code using IDE and eye tracking. In *IEEE Symp. on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 40–46, 2024.

[35] Joel S. Warm, Raja Parasuraman, and Gerald Matthews. Vigilance requires hard mental work and is stressful. *Human Factors*, 50(3):433–441, 2008.

[36] Christopher D. Wickens. Multiple resources and mental workload. *Human Factors*, 50(3):449–455, 2008.

[37] Robert M. Yerkes and John D. Dodson. The relation of strength of stimulus to rapidity of habit-formation. *Journal of Comparative Neurology and Psychology*, 18(5):459–482, 1908.